

1. Project Overview

This project implements a smart home controller using an Arduino Uno as the main processing unit. The system monitors environmental conditions using a temperature sensor, a light sensor, and a laser distance sensor, and presents information on a 16×2 LCD display. The controller operates in two distinct modes: Normal Mode, which functions as a home dashboard displaying temperature and time, and Security Mode, which actively monitors for suspicious activity such as sudden light changes or nearby movement. When a suspicious event is detected, a formatted alert is printed to the Serial monitor. Mode switching is handled by a physical pushbutton.

2. Hardware Components

- Arduino Uno
- 16×2 LCD display (parallel, 4-bit mode)
- TMP36 analog temperature sensor
- LDR (light-dependent resistor / photoresistor)
- VL53L0X laser time-of-flight distance sensor
- Momentary pushbutton
- Green LED
- Red LED
- 10kΩ resistor × 2 (LDR voltage divider pull-down)
- 10kΩ potentiometer (LCD contrast)
- 220Ω resistor × 2 (LED current limiting)
- Breadboard, jumper wires

3. Wiring Description

LCD 16×2 (parallel 4-bit) The LCD is wired in 4-bit mode. VSS connects to GND and VDD to 5V. The VO contrast pin connects to the wiper of a 10kΩ potentiometer whose outer legs go to 5V and GND. RW is tied to GND for write-only operation. RS connects to D7, EN to D6, and data lines D4–D7 of the LCD connect to Arduino pins D5, D4, D3, and D2 respectively. The backlight anode connects to 5V through a 220Ω resistor and the cathode to GND.

TMP36 Temperature Sensor The left leg (flat side facing you) connects to 5V, the right leg to GND, and the middle output leg connects directly to A0. No external resistor is required.

LDR One leg connects to 5V. The other leg connects to A1 and also to a 10kΩ resistor whose other end goes to GND, forming a voltage divider. Higher light levels produce a higher voltage at A1.

VL53L0X Laser Distance Sensor VIN connects to the Arduino's 3.3V pin. GND connects to GND. SDA connects to A4 and SCL to A5, using the Arduino's hardware I2C bus. XSHUT and GPIO1 are left unconnected.

Pushbutton One leg connects to D8 and the other to GND. The Arduino's internal pull-up resistor is enabled via `INPUT_PULLUP`, so no external resistor is needed. A press pulls D8 LOW.

LEDs The green LED anode connects through a 220Ω resistor to D9, with its cathode to GND. The red LED anode connects through a 220Ω resistor to D10, with its cathode to GND.

4. Photos of Completed Circuit

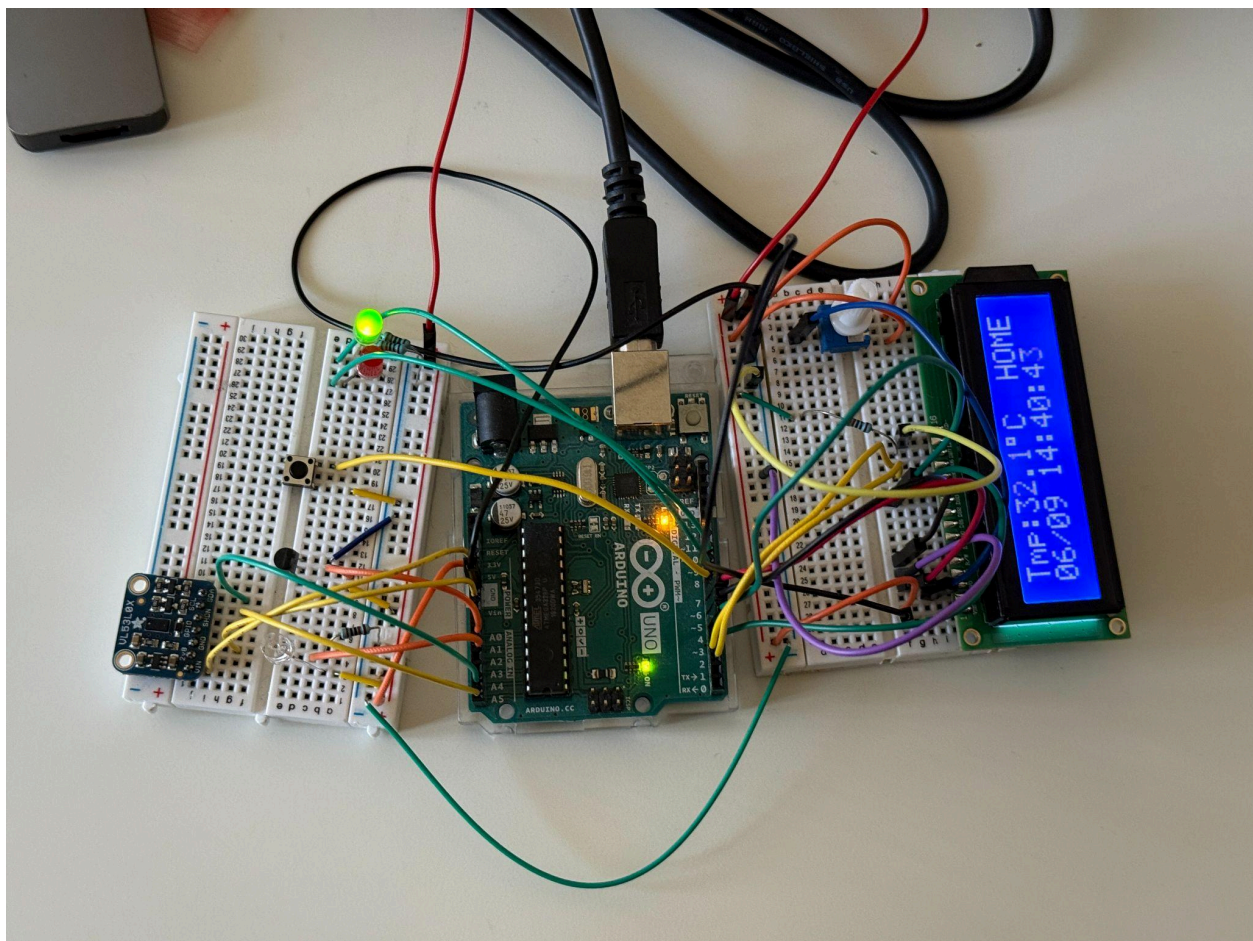


Figure 1: Smart home controller circuit in normal mode. Note that the LCD displays the current temperature reading, date, and time. (The date and time are hard coded using `millis()` because there is no internet connection from the Arduino Uno).

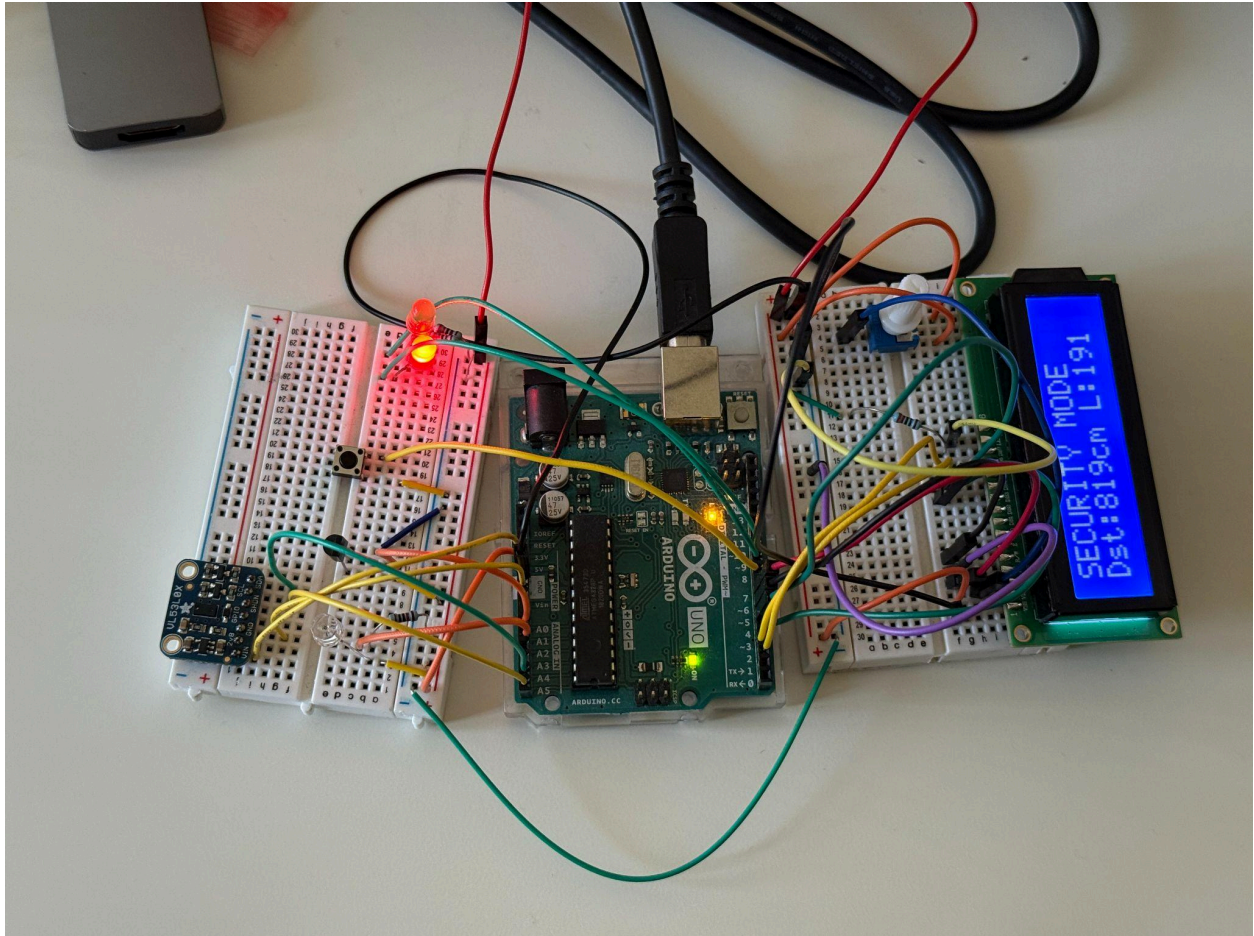


Figure 2: Smart home controller circuit in secure mode. Note that the LCD displays the distance and light readings.

5. Normal Mode

When the system starts, it enters Normal Mode. The green LED turns on steady to indicate the system is active and the user is home. The LCD displays the current temperature on the top row alongside a "HOME" label. The bottom row shows the current date and time, derived from the compile-time timestamp and incremented using `millis()` to track elapsed time since boot.

The temperature is read from the TMP36 by averaging 10 ADC samples and converting the result to Celsius using the TMP36's linear voltage formula: $\text{tempC} = (\text{voltage} - 0.5) \times 100$. The LCD refreshes once per second. If the temperature exceeds 35°C or drops below 10°C, a warning message replaces the date/time row on the LCD.

6. Security Mode

Pressing the button switches the system to Security Mode. The green LED turns off and the red LED turns on steady to indicate the armed state. The LCD displays "SECURITY MODE" on the top row and live sensor readings on the bottom row.

In this mode, the system polls the LDR and VL53L0X every 250ms. Two types of suspicious events are monitored. First, a significant change in light level — an absolute difference greater than 150 ADC counts from the last reading — is treated as a light event, indicating a light being turned on or off in the monitored area. Second, if the VL53L0X measures a distance of less than 40cm, a motion event is triggered. When either event fires, the red LED blinks six times, the LCD displays "!ALERT!", and a formatted alert message is printed to the Serial monitor. A 5-second cooldown prevents the same event from triggering repeated alerts. Pressing the button again returns the system to Normal Mode.

7. Sensor Usage

TMP36 — Connected to A0. The sensor outputs a voltage linearly proportional to temperature (10mV/°C, 750mV at 25°C). The Arduino reads the analog voltage and converts it to Celsius. Ten samples are averaged per reading to reduce noise introduced by breadboard power supply variation.

LDR — Connected to A1 in a voltage divider with a 10kΩ pull-down resistor. As light intensity increases, the LDR's resistance decreases, raising the voltage at A1. In Security Mode, the system stores a baseline reading when the mode is entered and compares each new reading against it to detect sudden changes in ambient light.

VL53L0X — Connected via I2C on A4 (SDA) and A5 (SCL). This sensor uses laser time-of-flight to measure distance to the nearest object in millimetres. The sketch reads a single-shot range measurement each poll cycle and converts it to centimetres. If the returned value is below the 40cm threshold, a motion event is triggered. If the sensor times out, the function returns -1 and no event is raised.

8. Notification Method

The project uses Serial monitor output as a substitute for internet-based email notification. When a suspicious event is detected in Security Mode, the `sendSerialAlert()` function prints a formatted message to the Serial port at 9600 baud. The message follows the format specified in the project brief, including the event type, sensor values for both the distance and light sensors, and a timestamp. The timestamp is computed from the compile-time `__DATE__` and `__TIME__` macros with elapsed milliseconds added to produce a running clock that updates in real time. A sample alert looks like this:

Subject: Arduino Uno Security Alert

Security Alert!

Mode: Security Mode

Event: Motion detected

Ultrasonic distance: 14 cm

Light sensor value: 720

Time: 2026-06-09 8:35 PM

9. Code Structure and Important Functions

The sketch is organised into distinct sections: pin definitions and constants at the top, followed by sensor reading functions, display functions, mode management functions, and finally `setup()` and `loop()`.

`readTemperature()` — Averages 10 ADC samples from A0, converts the result to a voltage using a calibrated reference, and applies the TMP36 formula to return temperature in Celsius.

`readLDR()` — Returns a single raw ADC reading from A1.

`readDistanceCm()` — Calls the Pololu VL53L0X library's `readRangeSingleMillimeters()`, checks for a timeout, and returns the distance in centimetres or `-1` on failure.

`formatCurrentDateTime()` / `formatEmailDateTime()` — Compute the current date and time by parsing the compiler's `__DATE__` and `__TIME__` macros and advancing the result by the number of seconds elapsed since boot via `millis()`. The two functions format the result differently: one for the LCD (compact) and one for Serial alerts (human-readable with AM/PM).

`enterNormalMode()` / `enterSecurityMode()` — Handle all state changes associated with a mode switch: LED states, LCD clear, LDR baseline capture, and cooldown timer reset.

`updateNormalDisplay()` — Called every second in Normal Mode. Reads temperature, formats the date/time, and writes both rows of the LCD.

pollSecuritySensors() — Called every 250ms in Security Mode. Reads the LDR and VL53L0X, checks both event conditions with cooldown guards, updates the LCD, blinks the LED, and calls **sendSerialAlert()** if an event is detected.

buttonPressed() — Implements software debouncing with a 50ms hold-off timer. Returns true once per confirmed LOW transition on the button pin.

sendSerialAlert() — Prints the formatted security alert message to the Serial monitor with event type, sensor values, and timestamp.

sendDashboardData() — Emits a single comma-separated **DATA, . . .** line once per second containing mode, temperature, light, and distance values for external consumption.

loop() — Runs three concurrent periodic tasks using non-blocking **millis()** comparisons: LCD refresh (1000ms), security sensor polling (250ms), and dashboard data output (1000ms). Button state is checked on every iteration.

10. Testing Results

Temperature sensor — At room temperature (~22°C), the TMP36 initially read approximately 52°C due to USB bus voltage being lower than the assumed 5V reference. Adjusting **vRef** to 3.5V in the conversion formula brought the reading to a stable and accurate room temperature value. Averaging 10 samples eliminated the occasional single-sample spikes observed during initial testing.

LDR — Covering the LDR with a hand in Security Mode reliably triggered the light-change event. Turning a desk lamp on and off in the vicinity of the sensor produced clean, repeatable triggers. The 5-second cooldown prevented alert flooding during gradual light transitions.

VL53L0X — The sensor successfully detected an object placed within 40cm and returned to a non-triggering reading once the object was removed. Readings beyond the sensor's effective range (~120cm for the default timing budget) returned timeout values, which were handled gracefully by the -1 fallback.

Button / mode switching — The debounce logic performed reliably across many press cycles with no false triggers observed. Transitions between Normal and Security Mode were immediate, with the LDR baseline correctly captured on each entry to Security Mode.

LCD — All display content rendered correctly within the 16-character line limit. The degree symbol rendered correctly using character code 223. The running clock advanced accurately over a 10-minute observation period.